

PROJECT IMPLEMENTATION PLAN

AI Stock Intelligence Platform

Confidence-Aware Decision Support System for NIFTY 50 and Key Sectoral Indices

6-Week Development Roadmap | 4 Parallel Tracks | End-to-End AI/ML Pipeline

TEAM SIZE	DURATION	CORE MODEL	PLATFORM
4 Members	6 Weeks	LSTM + MC Dropout	React + FastAPI (Local)

Abstract

This document is the working implementation plan for building an AI-based Stock Intelligence Platform covering NIFTY 50 stocks plus key sectoral indices (Nifty Bank, Nifty IT, Nifty Next 50). The system fetches historical price data, market context, and financial news; engineers 100+ quantitative features; trains an LSTM model with Monte Carlo Dropout for uncertainty-aware return forecasting; and serves confidence-calibrated Buy / Hold / Avoid signals through a locally-run React dashboard backed by a FastAPI service. Work is organized into four independent, parallel tracks rather than a single sequential pipeline, so all four members can work at the same time every week with only a small number of clearly flagged hand-off points.

Parallel Tracks at a Glance

Member	Track	What keeps it independent
M1	Price & Market Data	Owns fetching + technical features end to end; needs no one else's output.
M2	News & Sentiment	Owns news fetching + sentiment scoring end to end; needs no one else's output.
M3	Modeling & Evaluation	Builds and tunes the model on sample data first; switches to real features at one agreed checkpoint.

Member	Track	What keeps it independent
M4	Product & Dashboard	Builds the API and UI against mock data first; switches to real model output at one agreed checkpoint.

SECTION 1

Feature Overview

What the platform actually does, end to end

1.1 Core Capabilities

#	Feature	Description
1	Automated Data Collection	Pulls OHLCV price data, index data, and news headlines with no manual downloads.
2	100+ Feature Engineering	Auto-generates technical indicators, lag features, market-context and fundamental columns from raw price data.
3	News Sentiment Analysis	Converts financial headlines into numerical sentiment scores using transformer-based NLP.
4	Multi-Horizon Forecasting	Predicts expected return for next day, next week, next month, and next 6 months.
5	Confidence-Aware Prediction	Uses Monte Carlo Dropout to attach a confidence score to every prediction.
6	Rule-Based Signal Engine	Combines return, confidence, sentiment and technicals into Buy / Hold / Avoid labels.
7	Backtesting & Validation	Tests the model against unseen historical data and a random-strategy baseline.
8	Confidence Calibration Report	Checks whether stated confidence matches real-world correctness rate.
9	Explainability (SHAP)	Shows which features drove a specific Buy/Hold/Avoid call, not just the output number.
10	Local Interactive Dashboard	React front-end with price charts, indicators, news, sentiment, signals and backtest views, powered by a local FastAPI service.

1.2 Expanded Market Coverage

The platform is not hard-locked to a single 50-stock list. The data-fetch and feature layers are ticker-agnostic by design, so the covered universe is expanded beyond NIFTY 50 to demonstrate generalization across sectors and market-cap tiers, without adding new architectural complexity.

Universe	What it adds	Purpose
NIFTY 50	Core large-cap universe, primary training/testing set	Main dataset for model development and demo
Nifty Bank	Banking-sector index and constituents	Tests sector-specific behavior (rate-sensitive stocks)
Nifty IT	IT-sector index and constituents	Tests behavior of export-driven, currency-sensitive stocks
Nifty Next 50	Next tier of large-caps	Generalization test on stocks the model was not tuned on

1.3 Differentiator Add-Ons

Beyond a standard college LSTM-predicts-price project, this plan bakes in the features below from week one, because they are cheap relative to their impact on grading and demo quality.

Add-on	Why it matters	Track
SHAP Explainability	Shows why a signal was generated, not just what it is.	M3
Confidence Calibration Report	Shows the model's stated confidence lines up with real outcomes.	M3
Ensemble Disagreement (LSTM + XGBoost)	Flags conflicting signals instead of forcing false certainty.	M3
Sentiment-Price Divergence Alerts	Flags cases where price and sentiment disagree.	M2
Regime Detection	Adjusts confidence based on trending / choppy / volatile market state.	M1
Daily 'What Changed' Digest	Auto-generated plain-English summary of signal changes day over day.	M4

Technology Stack

Latest stable, well-supported tools — chosen for reliability, not novelty

2.1 Core Stack

Layer	Technology	Purpose
Language (backend)	Python 3.12	Data pipeline, ML, and API implementation.
Language (frontend)	TypeScript + React 18 (Vite)	Local dashboard UI, component-based and fast to build.
Backend API	FastAPI	Local REST service that serves processed data, predictions and signals to React.
Data Handling	Pandas 2.x, NumPy 2.x	Data wrangling, cleaning, merging multi-source datasets.
Price Data	yfinance	Free OHLCV data for stocks and indices across all covered universes.
Technical Indicators	pandas-ta	Auto-generates 100+ indicator columns from OHLCV in one call.
News Data	NewsAPI / Finnhub / GNews	Recent financial headlines per stock ticker.
NLP Sentiment	FinBERT (Hugging Face Transformers)	Finance-tuned sentiment scoring, more accurate than generic tools.
Deep Learning	PyTorch 2.x	LSTM model training, MC Dropout inference, optional GPU support.
Classical ML (ensemble)	XGBoost	Tabular model for ensemble-disagreement cross-check.
Explainability	SHAP	Feature attribution for individual predictions.
Evaluation	scikit-learn, SciPy	Regression/classification metrics, statistical tests.
Charting (frontend)	Recharts / Plotly.js	Interactive charts rendered inside the React dashboard.
Storage	SQLite + Parquet	Lightweight local storage for raw and processed datasets.
Experiment Tracking	MLflow (local)	Logs training runs, metrics, and model versions.
Version Control	Git + GitHub	Shared codebase, one branch per track, pull-request reviews.

2.2 Why React + FastAPI Instead of Streamlit

The dashboard runs fully locally: a FastAPI service exposes JSON endpoints for prices, indicators, sentiment, predictions and signals, and a React app consumes those endpoints. This keeps everything on the team's own machines with no external hosting dependency, gives full control over UI/UX, and is closer to how production analytics tools are actually built.

2.3 Model Architecture Summary

Component	Choice	Reasoning
Sequence Model	Stacked LSTM (2 layers, 64-128 units) + Dropout	Captures temporal dependency in 60-90 day rolling windows; dropout enables MC Dropout inference.
Uncertainty Estimation	Monte Carlo Dropout (30-50 stochastic passes)	Established way to get prediction confidence without training a second model.
Cross-check Model	XGBoost Regressor on flattened tabular features	Independent model family; disagreement with LSTM signals genuine uncertainty.
Sentiment Model	FinBERT (pre-trained, fine-tune optional)	Domain-tuned for financial text; clear accuracy gain over generic sentiment tools.
Explainability	SHAP DeepExplainer / KernelExplainer	Attributes each prediction to contributing features for transparency.
Signal Layer	Rule-based decision engine (return + confidence + sentiment + technicals)	Transparent, explainable logic instead of a black-box classifier for the final label.

2.4 Datasets — All Derived From Free, Public Sources

No pre-made '100-indicator' dataset needs to be found anywhere. Raw OHLCV data is transformed into 100+ feature columns using pandas-ta, lag features, and merged market/fundamental context, across the full expanded universe described in Section 1.2. All sources below are free and require no paid API keys for the project's scope.

Dataset	Source	Granularity	Used For
NIFTY 50, Nifty Bank, Nifty IT, Nifty Next 50 stock OHLCV	Yahoo Finance (yfinance)	Daily, 5+ yrs	Core price data, indicators, LSTM input
Corresponding index levels	Yahoo Finance	Daily	Market/sector-wide context feature
India VIX	Yahoo Finance	Daily	Volatility / risk context feature
USD/INR, Crude Oil	Yahoo Finance	Daily	Macro context features
Financial news headlines	NewsAPI / Finnhub / GNews	Real-time / daily	Sentiment scoring input
Company fundamentals	yfinance (info module)	Static / quarterly	P/E, market cap, sector as extra features

SECTION 3

How the Team Works in Parallel

Four independent tracks, one shared interface contract

Instead of a single pipeline where each person waits on the previous stage (data → features → model → dashboard), work is split into four vertical tracks that each own their piece end to end. Every member can start working in Week 1 without needing anyone else's output, because the team agrees on a shared data schema up front and everyone who needs another track's output builds against sample or mock data matching that schema until the real thing is ready. Swapping sample data for real data at that point is a drop-in replacement, not a redesign.

3.1 The Interface Contract (agreed in Week 1, Day 1)

Contract	Defined by	Used by
Raw price/index schema (columns, date format, file layout)	M1	M2, M3, M4 build against this shape using sample data
Sentiment score schema (per ticker, per date)	M2	M3, M4 build against this shape using sample data
Model output schema (return, confidence, signal per horizon)	M3	M4 builds its API/UI against this shape using mock data

3.2 The Four Tracks

M1 Price & Market Data

Fully self-contained: fetches and processes all price-based data end to end. Never waits on another track.

- Yahoo Finance / yfinance integration across NIFTY 50, Nifty Bank, Nifty IT, Nifty Next 50
- Index/VIX/macro data collection
- Data cleaning & date alignment
- 100+ technical indicator generation (pandas-ta) and lag features
- Regime detection (volatility clustering)

M 2

News & Sentiment

Fully self-contained: fetches headlines and produces sentiment scores end to end. Never waits on another track.

- News API integration (NewsAPI/Finnhub/GNews)
- Headline cleaning & storage
- Sentiment scoring pipeline (TextBlob baseline → FinBERT)
- Sentiment-price divergence logic (merges its own sentiment with M1's already-finished price trend near Week 3)

M 3

Modeling & Evaluation

Builds and tunes the model against public sample data for the first three weeks; switches to the real merged feature set at one agreed checkpoint in Week 4.

- LSTM architecture design & training on sample data first
- Monte Carlo Dropout confidence estimation
- XGBoost ensemble cross-check model
- SHAP explainability integration
- Confidence calibration & generalization testing

M 4

Product & Dashboard

Builds the API and UI against mock JSON matching the agreed schema for the first three weeks; switches to real model output at one agreed checkpoint in Week 4.

- FastAPI backend service (JSON endpoints)
- React dashboard (all pages/components)
- Rule-based Buy/Hold/Avoid signal engine (logic is independent of whose numbers feed it)
- Backtesting & random-baseline comparison
- Daily 'what changed' digest

Across the entire 6 weeks, there are only **two** points where a track genuinely needs another track's real output rather than its own sample/mock data — both are flagged clearly in Section 4 where they occur (Week 3-4). Every other week, all four members can work fully in parallel.

SECTION 4

6-Week Execution Plan

Each member's tasks per week — designed to run in parallel

WEEK 1

Independent Kickoff — zero cross-track dependencies

Weekly Goal: Every member starts building immediately. By end of week 1, all four interface-contract schemas are agreed and documented, and each track has a working first version using its own or sample/mock data.

M 1 *Price & Market Data*

- Set up GitHub repo, folder structure, environment (venv/conda)
- Build yfinance fetch module for OHLCV (5+ years) for pilot stocks from NIFTY 50 and Nifty Bank
- Fetch corresponding index levels & India VIX historical data
- Draft and publish the raw data schema (column names, formats) for the other tracks to build against

M 2 *News & Sentiment*

- Shortlist and test a news API (NewsAPI/Finnhub/GNews) and its rate limits
- Build headline-fetching module keyed by stock ticker
- Implement baseline TextBlob sentiment scoring on fetched headlines
- Draft and publish the sentiment schema for the other tracks to build against

M 3 *Modeling & Evaluation*

- Set up PyTorch environment (CPU/GPU), test install
- Download one public sample OHLCV series (any single stock) to prototype independently
- Build baseline single-layer LSTM on that sample data — no need to wait for real features
- Draft and publish the model input/output schema for M4 to build against

M 4 *Product & Dashboard*

- Scaffold FastAPI backend project structure and health-check endpoint
- Scaffold React app (Vite + TypeScript) with placeholder routes/pages
- Create mock JSON files matching the schemas M1/M2/M3 just published, and wire the first UI pages to them
- Draft the rule-based signal engine logic using the mock data (thresholds only, no real numbers needed yet)

Milestone: Checkpoint: 3 interface schemas (price, sentiment, model output) documented and shared; each track has its own working v1 using its own or sample/mock data. No one is blocked on anyone.

WEEK 2

Deepen Each Track — still fully independent

Weekly Goal: Each track goes from a working v1 to a near-complete version, entirely on its own data or sample/mock data.

M1 *Price & Market Data*

- Build data cleaning pipeline: missing values, date alignment, duplicates
- Extend collection to full NIFTY 50 + Nifty Bank + Nifty IT + Nifty Next 50
- Add USD/INR and crude oil as macro-context sources
- Generate 100+ indicator columns via `pandas-ta.strategy('All')` on the cleaned data

M2 *News & Sentiment*

- Extend headline collection to the full stock/index universe
- Begin FinBERT integration to replace the TextBlob baseline
- Build sentiment aggregation (daily average score per ticker)
- Write unit tests for the sentiment pipeline using saved sample headlines

M3 *Modeling & Evaluation*

- Extend the sample-data LSTM to multi-horizon output (1D, 1W, 1M, 6M)
- Add Monte Carlo Dropout inference on the sample-trained model
- Prototype the XGBoost ensemble model on the same sample data
- Write the data-loader code so it can accept the real feature files once M1/M2 are ready (not needed yet)

M4 *Product & Dashboard*

- Build out FastAPI endpoints returning mock data for indicators, sentiment and predictions
- Build the React price-chart and news/sentiment pages against those mock endpoints
- Fully implement and unit-test the signal engine logic against mock inputs
- Build the backtesting skeleton against mock historical signals

Milestone: Checkpoint: M1 has a real, cleaned, 100+ feature dataset. M2 has real sentiment scores. M3 and M4 each have a functionally complete module running end-to-end on sample/mock data. Still zero blocking between tracks.

WEEK 3

Finalize Real Data — prepare for the one real hand-off

Weekly Goal: M1 and M2 finalize their real datasets in the agreed schema. M3 and M4 keep building on sample/mock data, but their code is now ready to accept the real files as a drop-in swap.

**M
1**

Price & Market Data

- Finalize regime detection (volatility clustering) on the full dataset
- Produce the final real 'price_features' file/table matching the Week-1 schema exactly
- Automate a daily data-refresh script
- Support M2 only if a schema question comes up (rare, one-off)

**M
2**

News & Sentiment

- Finalize FinBERT sentiment pipeline, replacing TextBlob as default
- Produce the final real 'sentiment_features' file/table matching the Week-1 schema exactly
- Implement sentiment-price divergence logic, reading M1's already-finished price trend column
- Validate no data leakage between feature timestamps and prediction targets

**M
3**

Modeling & Evaluation

- Continue tuning architecture (layers, hidden units, dropout, sequence length) on sample data
- Finish the data-loader adapter so it can point at either sample data or the real files with one config change
- Prepare the training script to run unattended once pointed at real data
- No blocking work yet — real switch happens in Week 4

**M
4**

Product & Dashboard

- Continue building remaining React pages (overview, watchlist) against mock data
- Finish the API contract for how predictions will be exposed once real
- Add UI states for loading/error so swapping data sources later needs no UI rework
- No blocking work yet — real switch happens in Week 4

Milestone: Checkpoint: M1's and M2's real feature files exist and match the agreed schema exactly. M3 and M4's code is ready to consume them but hasn't needed to yet.

WEEK 4

The One Real Hand-Off — minimal, scoped, and flagged

Weekly Goal: This is the only week with genuine cross-track dependency. It is a drop-in data swap, not a redesign, because the schema was fixed in Week 1.

M 1 *Price & Market Data*

- Support M3's integration by validating the real feature files load correctly
- Keep improving pipeline performance and full-universe coverage in parallel
- Document the 100+ feature columns (data dictionary)

M 2 *News & Sentiment*

- Support M3's integration by validating the real sentiment files load correctly
- Keep improving news coverage and sentiment accuracy in parallel
- Add regime label (from M1) as an extra confidence modifier input to the signal engine spec

M 3 *Modeling & Evaluation*

- Swap sample dataset for M1 + M2's real merged features and retrain the LSTM
- Run Monte Carlo Dropout + build the confidence calibration report on real data
- Integrate SHAP (DeepExplainer) for per-prediction feature attribution
- Finalize the XGBoost ensemble and the LSTM-vs-XGBoost disagreement flag
- Run generalization test: evaluate on Nifty Next 50 stocks unseen in training

■ **CROSS-TEAM DEPENDENCY:** Needs M1's real price/feature file and M2's real sentiment file (both finalized in Week 3, same schema agreed in Week 1) to swap in for sample data.

M 4 *Product & Dashboard*

- Swap mock JSON for M3's real prediction output via the agreed model-output schema
- Wire the dashboard's signal cards, calibration chart and SHAP view to real data
- Add the ensemble disagreement warning banner to signal cards
- Keep building the remaining UI (digest, polish) in parallel — not blocked once the swap is done

■ **CROSS-TEAM DEPENDENCY:** Needs M3's real prediction output (schema agreed in Week 1) to swap in for mock JSON.

Milestone: Cross-team dependency window: M3 needs M1+M2's real files; M4 needs M3's real predictions. Both are same-day swaps against an already-agreed schema, not new integration work.

WEEK 5

Full-Scale Hardening — independent again

Weekly Goal: Each track scales and polishes its own piece across the full expanded universe.

M 1 *Price & Market Data*

- Run and validate the full data pipeline across the entire expanded universe
- Add graceful fallback for stocks with missing/incomplete data
- Performance-optimize storage reads for dashboard responsiveness

M 2 *News & Sentiment*

- Validate sentiment pipeline stability across all covered stocks/indices
- Handle edge cases: low-news stocks, illiquid stocks, corporate actions
- Finalize sentiment documentation and sanity-check score distributions

M 3 *Modeling & Evaluation*

- Retrain final models on the full dataset with best hyperparameters
- Freeze final model artifacts (weights, scalers, configs) with versioning
- Re-run calibration and backtest across the full expanded universe

M 4 *Product & Dashboard*

- Complete all React pages: overview, stock detail, signals, backtest, digest
- Add stock/index search, watchlist, and portfolio-view (ranked list)
- Polish UI/UX: layout, color coding for Buy/Hold/Avoid, loading states
- Basic cross-browser testing of the local React app

Milestone: Checkpoint: full expanded-universe coverage; dashboard feature-complete; models frozen for final evaluation. No new cross-track blocking introduced.

WEEK 6

Testing, Documentation & Final Presentation

Weekly Goal: Each member tests and documents their own track independently; one short joint session covers final end-to-end testing.

M 1 *Price & Market Data*

- End-to-end test of the data pipeline from a fresh environment
- Write the data collection & storage documentation section
- Fix any bugs found in own track during testing

M 2 *News & Sentiment*

- Validate final sentiment outputs against spot checks
- Write the news & sentiment documentation section
- Fix any bugs found in own track during testing

M 3 *Modeling & Evaluation*

- Finalize the evaluation report: MAE/RMSE/MAPE/R2, directional accuracy, calibration
- Write the model architecture & evaluation documentation section
- Prepare model-related slides (architecture diagram, results)

M 4 *Product & Dashboard*

- Final QA pass on the React dashboard; fix UI bugs; package to run locally (npm build + unicorn)
- Assemble the final report/PDF and slide deck with all four tracks' contributions
- Rehearse the live demo script and Q&A; responses
- Prepare a backup demo video in case of live-demo issues

Milestone: Checkpoint: one joint end-to-end dry run of the live demo completed before presentation day.

Evaluation Plan & Expected Output

How success is measured, and what the final demo shows

5.1 Evaluation Metrics

Category	Metric	Purpose
Regression	MAE, RMSE, MAPE, R^2	How close predicted returns are to actual returns.
Directional	Directional Accuracy	Whether the model gets the up/down call right.
Validation	Backtest return vs random baseline	Whether the model performs better than chance.
Trust	Confidence Calibration	Whether stated confidence matches real-world correctness rate.
Generalization	Nifty Next 50 hold-out test	Whether the model performs reasonably on stocks outside its core training universe.

5.2 Final Expected Output

- Stock/index price chart with overlaid technical indicators
- Latest news headlines with sentiment scores
- Multi-horizon predicted returns (1D / 1W / 1M / 6M)
- Confidence score per prediction, with calibration report
- SHAP-based explanation for each Buy/Hold/Avoid signal
- Ensemble disagreement flag when LSTM and XGBoost diverge
- Backtest performance vs random-strategy baseline
- Daily 'what changed' digest summarizing signal shifts
- Coverage across NIFTY 50, Nifty Bank, Nifty IT, and Nifty Next 50

5.3 Risk & Mitigation

Risk	Mitigation
News API rate limits / downtime	Cache headlines locally; fall back to last-known sentiment; support multiple API providers.
LSTM training instability	Fix random seeds, use early stopping, checkpoint best model, keep XGBoost as a sane fallback.
Time overrun near deadline	Milestones checked weekly; Week 6 reserved purely for testing/docs, not new features.
Data leakage in features	Time-based train/test split only; strict validation that lag features never use future data.

Risk	Mitigation
Week 4 hand-off slips	Schema fixed in Week 1 and re-validated in Week 3, so the Week 4 swap is mechanical, not a design discussion.
Expanded universe adds data-quality edge cases	Graceful fallback for low-liquidity/low-news stocks; document any excluded tickers.

Prepared as a working implementation reference for the AI Stock Intelligence Platform project team.